

PERETTO & CO

MVP Fluxos Agênticos de AI

Arquitetura de Agentes · Modelos Gratuitos · Orquestração
Assíncrona

Documento atualizado com a hierarquia de modelos free
e arquitetura de subagents para operação CSM

Autor: Marcos
Código/Versão: 002/03
Área: CSM / Account Manager
Data: 17/05/2026
Periodicidade: A cada 1 mês
Status: Ativo

Sumário

1. Objetivo e Visão Geral
2. Hierarquia de Modelos Gratuitos
3. Arquitetura de Agentes
4. Pesos e Contrapesos
5. Agentes por Função
6. Fluxos de Orquestração Assíncrona
7. Transição AM → CSM
8. Framework Mínimo Viável
9. Integração com Ekyte
10. Instalação e Setup
11. Impacto Esperado

1. Objetivo e Visão Geral

Este documento apresenta o MVP de Fluxos Agênticos de AI da Peretto & Co — a primeira implementação prática de uma infraestrutura de agentes de AI rodando sobre a operação real de um squad de marketing de performance, agora com a transição estratégica de Account Manager para Customer Success Manager.

Não é estudo de caso. Não é piloto. É uma operação em produção, construída módulo a módulo, validada no Squad Prime, e projetada para replicar para toda a rede V4 Company.



O problema que este MVP resolve

Squads de marketing de performance tomam decisões baseadas em dados que chegam tarde, fragmentados e sem contexto. O gestor de tráfego descobre o problema na segunda quando o cliente já ligou na sexta. O account manager monta a ata depois da reunião, quando a memória já falhou. O coordenador percebe o desvio de OKR no replanejamento trimestral.

O resultado é tempo gasto em coleta e formatação de informação em vez de em decisão e execução.

Antes	Com o MVP
Briefing de comitê montado na hora da reunião	Briefing entregue domingo 20h, pace vs OKR por cliente
Ata preenchida depois, da memória	Transcrição automática + agente gera ata no formato padrão
Desvio de KPI percebido quando o cliente reclama	Detector de flags roda quinta 7h, antes do Growth
OKRs atualizadas quando alguém lembra	KRs calculados automaticamente toda quinta 8h
FCA aberta dias depois do problema	FCA gerada automaticamente quando anomalia é detectada
Um modelo para tudo	12 agentes com modelos especializados por função

2. Hierarquia de Modelos Gratuitos

Todos os modelos abaixo são **100% gratuitos** — via OpenCode Zen, Google AI ou OpenRouter. A hierarquia segue o princípio de usar o modelo certo para a tarefa certa, maximizando qualidade sem gastar um centavo.

Regra de ouro: O custo de raciocínio ruim é maior que o custo de token. DeepSeek V4 Flash para análise profunda, Gemini para visual, GPT-OSS para orquestração. Cada modelo no seu nicho.

Tier S — Modelos Principais

Modelo	Provedor	Contexto	Força	Agentes
S <code>opencode/deepseek-v4-flash-free</code>	OpenCode Zen	1M tokens	Raciocínio, código, análise complexa, matemática, lógica	@analista-dados, @revisor, @flag-*, @executor-comite
S <code>google/gemini-2.5-flash</code>	Google AI	1M tokens	Criação visual, HTML/CSS, multimodal, documentos, design	@gerar-pdf, @gerar-ppt, @gerar-html, @gerar-doc

Tier A — Orquestração e Escrita

Modelo	Provedor	Contexto	Força	Agentes
A <code>openrouter/openai/gpt-oss-120b:free</code>	OpenRouter	128k	Escrita geral, planejamento, orquestração CSM	@csm-orquestrador
A <code>openrouter/nousresearch/hermes-3-llama-3.1-405b:free</code>	OpenRouter	32k	Raciocínio profundo (405B), nuance, segunda opinião	Revisão crítica, análise de contrato

Tier B — Velocidade e Estrutura

Modelo	Provedor	Contexto	Força
--------	----------	----------	-------

B	opencode/minimax-m2.5-free	OpenCode Zen	256k	Velocidade, tarefas rápidas, classificação, parsing
B	opencode/nemotron-3-super-free	OpenCode Zen	128k	Output estruturado, schemas, formatação, extração
B	openrouter/nvidia/nemotron-3-super-120b-a12b:free	OpenRouter	128k	120B param, tarefas técnicas complexas

Tier C — Backup e Tarefas Leves

Modelo	Provedor	Contexto	Força
C opencode/qwen3.6-plus-free	OpenCode Zen	128k	Auxiliar leve, análises simples, backup geral
C openrouter/meta-llama/llama-3.3-70b-instruct:free	OpenRouter	128k	Backup confiável quando os S/A rate-limitarem
C openrouter/deepseek/deepseek-v4-flash:free	OpenRouter	1M	Mesmo modelo S via OpenRouter (fallback do Zen)

Tier D — Triviais

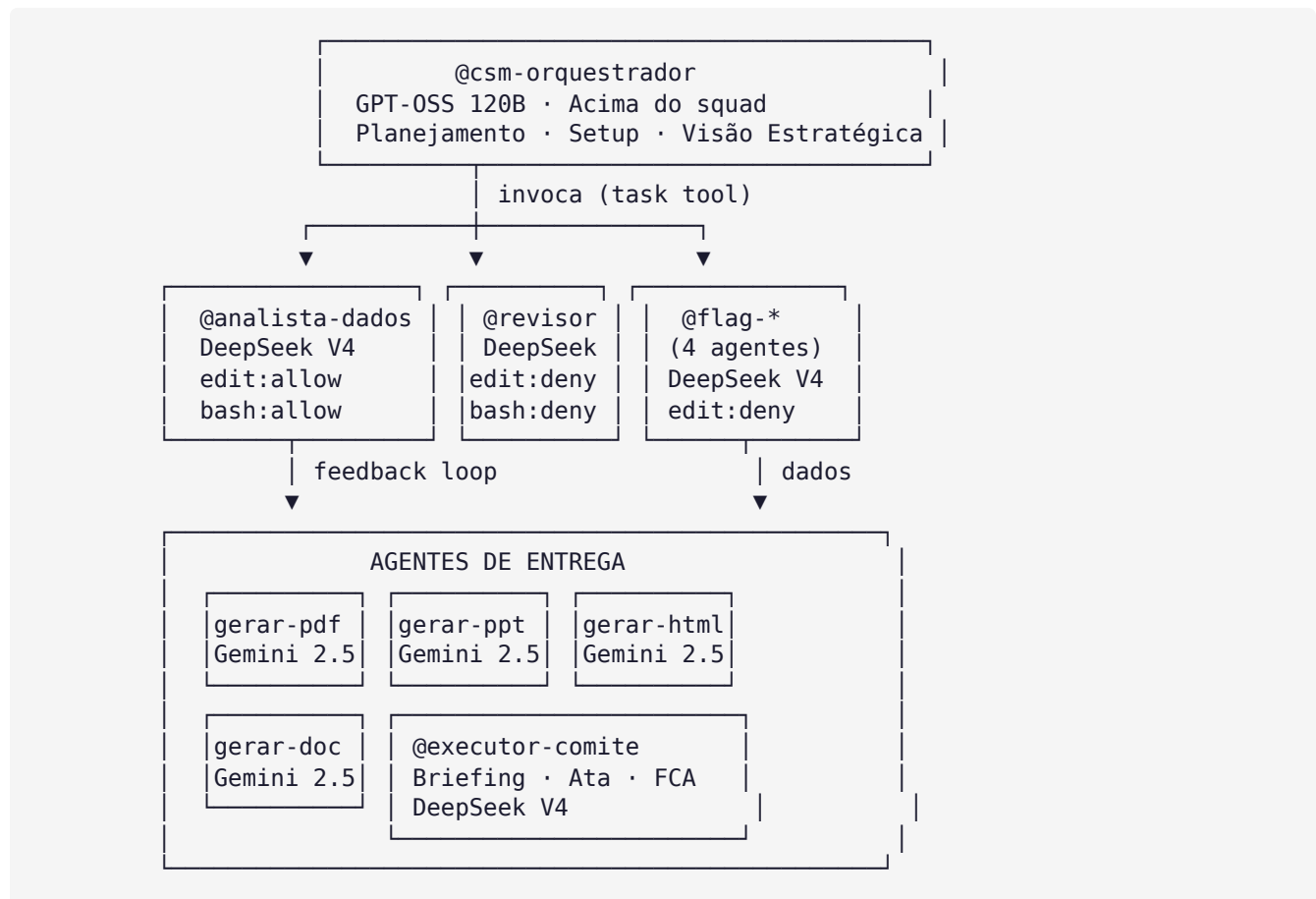
Modelo		Provedor	Contexto	Força
D	openrouter/google/gemma-4-31b-it:free	OpenRouter	8k	Classificação simples, regex, transform rápido
D	openrouter/nvidia/nemotron-3-nano-30b-a3b:free	OpenRouter	128k	Tarefas mínimas, extração simples

Estratégia de uso: DeepSeek V4 Flash (Zen) é o padrão do projeto. Gemini 2.5 Flash entra para criação visual/documentos. GPT-OSS 120B para orquestração CSM. Os tiers B/C/D são para tarefas específicas ou fallback. OpenRouter free serve como backup quando o Zen rate-limitar.

3. Arquitetura de Agentes

O OpenCode permite dois tipos de agente: **Primary** (alterna com Tab: Build/Plan) e **Subagent** (invocado com @). Criamos 12 subagents especializados, cada um com modelo, permissões e prompt específicos.

Diagrama de Camadas



4. Pesos e Contrapesos

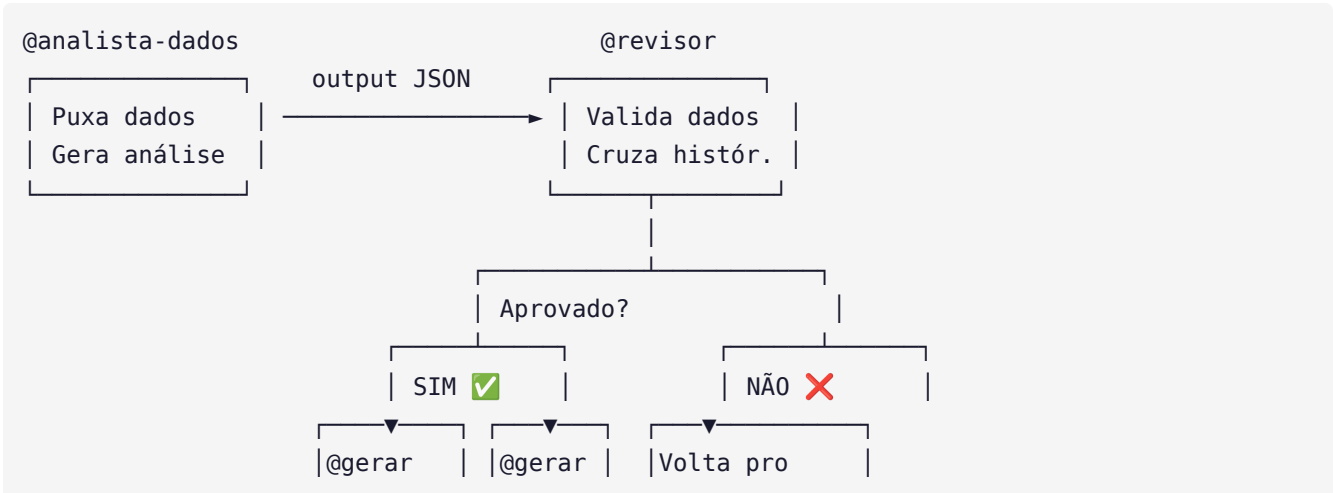
Matriz de Permissões por Agente

Agente	read	edit	bash	webfetch	Autonomia
@analista-dados	✓	✓	✓	✓	Relatórios internos
@revisor	✓	✗ deny	✗ deny	✓	Só valida, nunca edita
@gerar-pdf/ppt/html/doc	✓	✓	✓	✗	Documentos (sem revisão)
@flag-* (4 agentes)	✓	✗ deny	✓	✗	Diagnóstico, não execução
@csm-orquestrador	✓	✓	✓	✓	Setup inicial (com revisão)
@executor-comite	✓	✓	✓	✓	Briefings (com @revisor)

Regras de Ouro da Governança

- 1. **Agente que escreve em sistema real** (Ekyte, CRM, campanhas) → precisa de @revisor antes
- 2. **Agente que só cria documentos/relatórios** → roda autônomo sem revisão
- 3. **Agente que detecta anomalia** → só diagnostica, não executa. Quem executa é o humano ou script com crontab
- 4. **@revisor tem edit:deny** — ele aponta o erro, não corrige. Quem corrige é o agente executor

Fluxo de Validação Típico



-ppt/pdf	-html	@analista
-doc		com feedback

5. Agentes por Função

5.1 Agentes de Entrega (Gemini 2.5 Flash)

Agente	Função	Input	Output
@gerar-pdf	Gera PDFs estilizados padrão V4	"Relatório do cliente X"	.pdf + preview
@gerar-ppt	Apresentações para comitê/QBR/Growth	"Deck do comitê de segunda"	.html ou .pptx
@gerar-html	Dashboards e landing pages	"Dashboard de OKRs"	.html responsivo
@gerar-doc	Documentos, atas, propostas, FCAs	"Ata do Growth de terça"	.md → .docx/.pdf

5.2 Agentes de Análise (DeepSeek V4 Flash)

Agente	Função	Input	Output
@analista-dados	Análise multi-fonte, insights, JSON estruturado	"Analisa performance do cliente Y"	JSON com métricas + flags
@revisor	Valida outputs, confere números, formatação	"Revisa este relatório"	✅❌ com correções
@executor-comite	Briefing automático do comitê domingo 20h	"Prepara o comitê"	Briefing .md + .html

5.3 Agentes CSM

Agente	Modelo	Função
@csm-orquestrador	GPT-OSS 120B	Setup da unidade, triagem de flags, QBR, orquestração estratégica
@flag-roi	DeepSeek V4	ROAS abaixo da meta 2 semanas → classifica tipo + gera CHAS
@flag-churn	DeepSeek V4	NPS + CSAT caem → distingue percepção vs resultado
@flag-okr	DeepSeek V4	KR < 60% do esperado → execução vs premissa
@flag-operacao	DeepSeek V4	Sprint atrasa sem FCA → 3 níveis de severidade

6. Fluxos de Orquestração Assíncrona

Fluxo 1 — Preparação do Comitê de Segunda

Domingo 20h (cron ou comando manual):

1. @executor-comite → "Prepara o comitê de segunda"
2. @analista-dados (paralelo, sessões filhas por cliente)
 - [Cliente A] Puxa dados: OKR, ROAS, sprint, NPS
 - [Cliente B] Puxa dados: OKR, ROAS, sprint, NPS
3. @revisor (paralelo) → Valida cada JSON
4. @executor-comite consolida → briefing .md + .html
5. Usuário revisa na segunda 7h antes do comitê 8h

Fluxo 2 — Detecção de Flags (Quinta 7h)

1. @csm → "Roda detecção de flags para todos os clientes"
2. @flag-roi (paralelo por cliente) → verifica ROAS 2 semanas
3. @flag-churn (paralelo) → verifica NPS + CSAT
4. @flag-okr (paralelo) → verifica KRs
5. @flag-operacao (paralelo) → verifica sprints + timesheet
6. @csm consolida → prioriza por urgência → briefing ao CSM humano

Fluxo 3 — Preparação de QBR

Usuário: @csm "Prepara QBR do cliente X"

1. @csm → Invoca @analista-dados (quarter inteiro)
2. @analista-dados → ROAS q/q, OKR progress, sprints, NPS trend
3. @revisor → Valida dados
4. @gerar-ppt → Gera apresentação com layout V4
5. Usuário recebe o deck pronto

7. Transição AM → CSM

Baseado na Escola de CSM - Aula 1 e no documento Implementando o Manual de Operação.

O que muda

Hoje (Account Hero)	Novo (CSM Arquiteto)
Apaga incêndio operacional	Desenha estrutura que não precisa de bombeiro
Atende WhatsApp 22h	Filtra comunicação: time técnico blindado
Sobe campanha, faz criativo	Define objetivo de negócio, não o "como"
Reunião de 90min	Reunião de 30min com pauta fixa
Descobre problema quando cliente liga	Detector de flags roda antes do cliente sentir
Memoriza histórico	Vault Obsidian com memória persistente

Nova Divisão de Responsabilidades

CSM (@csm-orquestrador)
Foco: Resultado do cliente · ROI · Retenção · QBR
Acima do squad. Não executa – orquestra.
Ponto único de responsabilidade pelo resultado.
COORDENADOR
Foco: Operação · Qualidade técnica · Time Ops
Audita checklist · Facilita rituais · Protege o tambor
AM · GT · Copy · Design (Time Técnico)
Foco: Execução · Entrega · O "Como" técnico
Blindados pelo CSM – não falam com cliente direto

Timeline da Transição

Período	O que acontece
---------	----------------

Q2 2026 (Agora)	AM ainda opera como generalista. CSM agente em paralelo (setup + flags). Documentação da transição.
Q3 2026 (Início)	CSM assume como orquestrador. Coordenador assume operação técnica. AM vira executor no squad.
Q4 2026	CSM consolidado. Squad roda sem supervisão do CSM. CSM gerencia múltiplos squads.

8. Framework Mínimo Viável

Roadmap de Implementação

Step	O que	Status
1	Skills por função (builders-hub)	✔ Completo
2	Agentes OpenCode por função (12 subagents)	✔ Completo
3	Orquestração assíncrona (@ agente invoca @ agente)	✔ Configurado
4	Automações (cron + scripts + MCPs)	↺ Em progresso
5	Módulo Ekyte (leitura/escrita via API)	📋 Pendente

Estrutura de Módulo (por skill)

Cada função segue o padrão de 4 arquivos:

- **SKILL.md** — o agente em si (prompt + fluxos + templates)
- **CONTEXT.md** — o que o agente precisa saber sobre a operação
- **TRIGGERS.md** — quando usar, frases de ativação
- **OUTPUTS.md** — exemplos de output esperado para calibrar

9. Integração com Ekyte

O Ekyte tem API REST real em api.ekyte.com/v1.1 com autenticação por apiKey.

Módulos do Ekyte

Conector	Função
connector.py	Base de tudo. Autentica, pagina, cobre todos os endpoints.
checklist_ekyte.py	Substitui [?] do checklist por [S/N] reais do Ekyte.
sprints.py	Lê sprints ativas, calcula conclusão, gera relatório pro briefing.
fca.py	Cria FCAs automaticamente quando análise detecta desvio crítico.
base_conhecimento.py	Salva atas, análises e OKRs nos quadros do Ekyte.

10. Instalação e Setup

Pré-requisitos

- OpenCode instalado (última versão)
- Providers configurados: Google AI + OpenRouter
- OpenCode Zen ativado (modelos gratuitos disponíveis)
- Python 3.9+

Passo a Passo

1. Clone o repositório

```
git clone <seu-repo> && cd v4perettoco
```

2. Arquivos de configuração (já criados)

`opencode.json` — Config principal do projeto

`.opencode/agents/*.md` — 12 subagents

`~/.config/opencode/agents/*.md` — Espelho global

3. Instale dependências Python

```
pip install -r v4-automations/setup/requirements.txt
```

4. Configure .env

```
cp v4-automations/config/.env.template .env
```

Preencha: EKYTE_API_KEY, GOOGLE_OAUTH, META_ADS_TOKEN

5. Autentique Google OAuth

```
python v4-automations/setup/auth_google.py
```

6. Instale crons

```
python v4-automations/setup/install_cron.py
```

7. Teste um agente manualmente

No OpenCode TUI: `@analista-dados "testa conexão com dados mock"`

11. Impacto Esperado

Métricas

Métrica	Hoje	Meta 90 dias
Tempo de prep. do Comitê	45-60 min	< 5 min
Tempo para detectar desvio de KPI	3-5 dias	< 24h
Taxa de preenchimento de ata	~60%	> 95%
Tempo de abertura de FCA	1-3 dias	Automático
Atualização de OKRs	Manual semanal	Toda quinta 8h
Resposta a flag de churn	Semanas	48-72h
Onboarding de pessoa nova	1-2 semanas	< 2 dias

O que não é mensurável mas é mais importante

- Cada agente executando em paralelo = você faz 5 coisas ao mesmo tempo
- @revisor como contrapeso = autonomia sem perder controle
- Sessões assíncronas (sessão filha) = você navega entre tarefas sem perder contexto
- Sistema fica mais inteligente a cada semana: vault registra → OUTPUTS.md calibra → agente melhora
- Vantagem composta: quanto mais tempo opera com agentes, maior a distância de quem não tem

A janela de vantagem competitiva está aberta. Não por muito tempo. Quem construir a infraestrutura agora vai ser impossível de alcançar em três anos. O currículo de 2027 não vai perguntar qual linguagem você programa. Vai perguntar quantos agentes você sabe orquestrar.

V4 Company · Peretto & Co · MVP Fluxos Agênticos de AI · v002/03 · Maio 2026

Modelos Gratuitos: OpenCode Zen + Google AI + OpenRouter

Repositório: github.com/marcoslrusa/v4perettoco